

cks paths of dependency files.

- projectdependencies tracks package versions found in dependency files of projects.

sql

```
CREATE TABLE dependencyfiles AS (  
  id integer NOT NULL,  
  packagetype smallint NOT NULL,  
  path varchar NOT NULL  
);
```

```
CREATE TABLE projectdependencies AS (  
  id integer NOT NULL,  
  createdat timestamp without time zone NOT NULL,  
  dependencyfileid integer NOT NULL,  
  packageversionid integer NOT NULL  
);
```

Please note that in reality, package managers typically use two files to track project dependencies:

an editable dependency file, and a generated lock file,

but the Dependency Scanning analyzers and the report format they use only allow for one.

As a result, users might search for a path that isn't stored in the database, and get no results.

To address this, the Rails backend could convert the requested file path, or search for extra paths.

For instance, it could search for client/Gemfile.lock when users are searching for client/Gemfile.

In the future we could replace the dependencyfileid foreign key with a relation table to track all the files that contribute to a set of dependencies, or introduce a dependencyfilessets table that represents the sets.

By leveraging the dependencyfiles table, the backend can efficiently list project dependencies that match a given file path or package type.

The dependencyfiles table can be cleaned up automatically, similar to packages.

Since dependencyfiles records are coming from the scanning jobs and these can't be trusted, the default scope for the DependencyFile ActiveRecord model should only return objects where the path matches a supported dependency file found in the root directory:

Gemfile, Gemfile.lock, package.json, etc.

Custom filenames and dependency files found in sub-directories are excluded from the default scope.

Project dependencies are upserted when a new successful pipeline for the default branch generates a new set of Dependency Scanning reports.

The backend upserts into the following database tables in this order:

1. packages
1. packageversions

1. dependencyfiles
1. projectdependencies

If a scanning job is retried, then the backend reprocesses the corresponding pipeline as if this was a new one, to ensure data consistency.  
It goes through the following steps:

1. List all the completed jobs of the pipeline. Jobs that have been retried are excluded.
1. Collect the Dependency Scanning reports of these jobs.
1. Process these reports, and do the upserts.

All the DB tables involved in tracking project dependencies (packages, packageversions, dependencyfiles, and projectdependencies) are stored in the primary DB so that they can be efficiently joined when searching for specific packages across a namespace that has a large number of projects, using PostgreSQL. Moving project dependencies to a separate database would result in doing in-memory joins with too many project IDs.

Search across protected branches and releases

Users want to search project dependencies across all protected branches and releases.

Protected branches and project releases are particularly important because they usually correspond to:

- version branches under development or in maintenance mode
- releases being deployed and actively used

The names of the protected branches are stored in the protectedbranches.name column, and the git tags corresponding to the project releases are stored in releases.tag. (These columns already exist in the DB schema of GitLab.)  
Branch names and git tags are both git refs, and so we store a "ref" that can either be a git tag or a branch.

To avoid repeating the same git ref across all dependencies that belong to the same project branch or release, we introduce a new projectdependencysets table that groups dependencies by ref.

```
sql
CREATE TABLE projectdependencysets (
  id integer NOT NULL,
  projectid integer NOT NULL,
  ref character varying NOT NULL
);
```

Project dependencies now belong to dependency sets.

```
sql
```

```
ALTER TABLE projectdependencies ADD COLUMN projectdependencysetid NOT NULL;
```

Project dependencies no longer need to reference projects,  
and the projectid column is removed to save space.

sql

```
ALTER TABLE projectdependencies DROP COLUMN projectid;
```

When doing this change, we first create dependency sets for the default branches of the  
existing projects,

and then add the existing dependencies to these sets.

Only then we can add a NOT NULL constraint to the projectdependencysetid column;  
this cannot be done when creating the column.

The upsert of project dependencies is changed so that dependency sets are also upserted.

Navigate to scanning jobs

Users want to inspect the CI pipeline or CI job that reported a specific project dependency.

To keep track of the CI pipeline where a project dependency was detected,

we can leverage the projectdependencysets table used to group dependencies detected in the same  
branch or release;

this avoids repeating the same pipeline ID across many project dependencies.

sql

```
ALTER TABLE projectdependencysets ADD COLUMN pipelineid;
```

The pipelineid is set or updated when upserting project dependencies for a given git ref.

Alternatively, the concept of "dependency set" is changed, and a set now groups the  
dependencies detected by a single CI job.

This is more accurate, and users can be redirected to the exact job where a dependency was  
detected.

Also, this better supports the scenario where a CI job is retried, and the corresponding  
dependencies updated.

The upsert mechanism needs to be changed to upsert the multiple sets that correspond to the  
same ref.

Here are two ways this can be achieved:

- Track the job name in projectdependencysets.jobname,  
and upsert dependencies that correspond to a project ID, ref, and job name.
- Move the dependencyfileid from the projectdependencies table to projectdependencysets,  
and upsert dependencies that correspond to a project ID, ref, and file ID.

The latter is less coupled to the CI, and it results in a simpler logic where we upsert files,  
not files found in jobs.

When a branch is made protected, the last set of Dependency Scanning artifacts for that branch (coming from the latest pipeline) should be processed, and project dependencies should be added to the relational DB.

This way project dependencies for the newly protected branch can be queried without having to wait for a new pipeline.

Filter out development dependencies

Users want to exclude development dependencies when searching project dependencies.

They are not interested in the exact scope (Maven) or group (Bundler) these dependencies belong to.

A development dependency is a package that's only used when developing the project or its dependencies.

This might be package only used for testing, linting, or debugging the project or the libraries it uses.

Package managers and build tools use different concepts to track how dependencies are used:

- npm and yarn distinguish between development dependencies and others.
- Maven puts dependencies into 5 scopes, and each scope has well-defined semantics.
- Bundler supports user-defined dependency groups. They are naming conventions but the semantics of a particular group might be unknown.

For the purpose of the search, the exact dependency group/scope is converted to a boolean value,

and then stored into the development column of the projectdependencies table.

sql

```
ALTER TABLE projectdependencies ADD COLUMN development boolean;
```

projectdependencies.development is NULL when we can't establish whether this is a development dependency:

- The files processed by the analyzer don't provide the information.
- The semantics of the dependency group is unknown.

The exact dependency group or scope can be fetched from the reports and artifacts if/when needed.

In the future, if users need to filter project dependencies by dependency group, then the following changes are needed:

- Create a DB table that tracks dependency groups without repeating their names. A group has name and package type, so that its name can be interpreted in the context of the package type.
- Add a dependencygroupid foreign key to the projectdependencies table, so that the backend can track the group of a project dependency. This column is NULL when the information is not available.
- Drop the existing development column, because it becomes redundant with the new dependencygroupid column.

```
sql
CREATE TABLE dependencygroups AS (
  id integer NOT NULL,
  packagetype smallint NOT NULL,
  name varchar NOT NULL
);
```

```
ALTER TABLE projectdependencies ADD COLUMN dependencygroupid integer;
ALTER TABLE projectdependencies DROP COLUMN development;
```

Currently the Gemnasium analyzers don't track whether a package is a development dependency. Most of the lock files and graph exports they parse don't provide the information, and this information can't be stored in the Dependency Scanning reports.

Search for dependencies using specific licenses

Users want to list project dependencies that use a specific license.

Licenses of project dependencies are already tracked in the softwarelicenses table. It's currently used in combination of softwarelicensepolicies to set up a policy, and allow or deny a license in the context of project.

softwarelicenses can be reused to track licenses detected in project dependencies. A relation table is needed because a package version can have multiple licenses.

```
sql
CREATE TABLE projectdependencysoftwarelicenses AS (
  projectdependencyid integer NOT NULL,
  softwarelicenseid integer NOT NULL
);
```

Please note that this model doesn't accurately track composite licenses. For instance, it doesn't make the distinction between "MIT and BSD" and "MIT or BSD". It's assumed that this distinction isn't needed when searching for dependencies matching a license.

By joining projectdependencies with projectdependencysoftwarelicenses and softwarelicensepolicies, it's possible to efficiently list dependencies with unapproved licenses.

To reduce storage, licenses that are no longer referenced by project dependencies or license policies should be removed. This can be done when updating or destroying dependencies and policies, or in batch mode in a periodic job.

If we start collecting information about public packages (outside of user pipelines), including the licenses they use, then projectdependencysoftwarelicenses might repeat information stored in other tables.

However, this redundancy eliminates risks of collisions when the dependency is a private package whose name matches a public package.

In that particular case, licenses of a project dependency can differ from licenses of the corresponding public package.

### Search for dependencies affected by specific vulnerabilities

Users want to list project dependencies that are affected by a given vulnerability.

A project dependency might be affected by multiple vulnerabilities but a vulnerability belongs to a dependency.

Since vulnerabilities are tracked in the `vulnerabilityoccurrences` table, we could add a `projectdependencyid` foreign key to link a vulnerability finding to a project dependency.

By adding that column, we can efficiently search for project dependencies matching a vulnerability, and efficiently group vulnerabilities per dependency.

`projectdependencyid` would be NULL when the vulnerability has been reported by Dependency Scanning.

sql

```
ALTER TABLE vulnerabilityoccurrences ADD COLUMN projectdependencyid;
```

However, this change would link a generic table used for all vulnerabilities to a table that is specific to Dependency Scanning.

If that's an issue, then we can load both project vulnerabilities and project dependencies in memory,

and leverage the vulnerability location to remove dependencies that don't match the requested vulnerability.

The backend already loads project vulnerabilities and project dependencies in memory, and joins them when serving the dependency list, in order to group vulnerabilities per dependency.

However, project dependencies for the default branch would be fetched from the relational DB, and not from a collection of Dependency Scanning reports.

### Search for newly added dependencies

Users want to list project dependencies that were recently added to the default branch or a protected branch.

Project dependencies detected in any given branch are upserted in the `projectdependencies` table.

The `createdat` column isn't touched when doing the upsert, so it can be used to filter on dependencies recently added.

### Dependency policies

Users want to prevent specific packages from being added as project dependencies. They might exclude specific package versions, or exclude the package altogether.

Package policies are tracked by introducing a `projectdependency_policies` relation table that combines a project ID, a package ID, a classification, and a version range. This table is similar to the existing `software_license_policies` table used to store license policies.

```
sql
CREATE TABLE projectdependency_policies (
  id integer NOT NULL,
  projectid integer NOT NULL,
  packageid integer NOT NULL,
  classification integer DEFAULT 0 NOT NULL,
  versionrange varchar DEFAULT "":varchar NOT NULL
);
```

We can leverage the relational DB to JOIN project dependencies with corresponding policies, based on the `projectid` and `packageid`. However, the policies returned by the SQL query might not be a match, and the Rails backend must evaluate the `versionrange` to establish whether the policy applies to the package version being used as a dependency.

`projectdependency_policies` could also be called `package_policies`. This would be more consistent with the existing `software_license_policies` table but wouldn't communicate that packages are used as project dependencies.

### Dependencies audit

Users want to know when specific packages were added as project dependencies.

Project dependencies detected in any given branch are upserted in the `projectdependencies` table.

The `createdat` column isn't touched when doing the upsert, so it reflects the time when a dependency was first detected.

If we need to know the exact commit where a project dependency was first introduced, then we need a new column in the `projectdependencies` table to store the SHA of that commit.

```
sql
ALTER TABLE projectdependencies ADD COLUMN firstcommitsha bytea NOT NULL;
```

The first commit could also be retrieved by storing the ID of the pipeline where a project dependency was first detected. However, we would need to access the CI database, and that should be avoided for performance reasons.

### Track changes in dependencies

When reviewing an MR, users want to review changes to project dependencies.

The Rails backend compares project dependencies between the source branch and the target branch,

and prepares a diff that's presented in the MR page.

This is similar to how newly-introduced and fixed vulnerabilities are presented in a MR.

Project dependencies being compared can be fetched from different data sources, and the Rails backend uses the fastest option that's available:

1. fetch from the relational DB, if has the project dependencies
1. else load from the internal Software of Bill of Materials (SBoM) if available
1. else load from the collection of Dependency Scanning reports

Project dependencies all respond to the same interface so that they can be compared in the same way, independently of the data source.

The internal SBoM (when supported) can be created dynamically when calculating the diff, acting as an intermediary cache.

The diff itself can also be cached to speed up the loading of the MR page.

#### Dependency confusion protection

Users want to be warned about possible dependency confusion attacks.

These should be reported as vulnerability findings listed in the security dashboard of their projects.

#### Reporting changes in package checksums

Users want to be warned when the checksum of a project dependency has changed even though the package name, package version, and package source (if tracked) haven't changed.

Package checksums need to be reported by Dependency Scanning analyzers as part of the Dependency Scanning report.

Once checksums are made available to the Rails backend, they can be stored in the projectdependencies table.

sql

```
ALTER TABLE projectdependencies ADD COLUMN checksum varchar;
```

Before upserting project dependencies when a CI pipeline finishes, the backend fetches the current checksums from the projectdependencies table and compares them to the checksums of the corresponding dependencies it has in memory (and about to be upserted).

It then creates vulnerability findings for each project dependency where the new checksum doesn't match the old one, and eventually proceeds with the upsert.

The security check can be handled in a separate job so that it doesn't impact the upsert.

Checksums can't be stored in the packageversions table because:

- A package version can have multiple distributions, and each distribution can have multiple checksums.
- A package distribution can have multiple checksums corresponding to different hashing algorithms, like MD5 and SHA1.
- packages and packageversions don't track the package source, and there might be a collision between a public package and a private one.

Since the backend simply compares checksums, it doesn't matter that projectdependencies doesn't track the package source.

Reporting a change of package source

Users want to be warned when the source of a project dependency has changed. In particular, they want to be notified when the package used to be fetched from a private registry, and is now fetched from a public registry.

Package sources need to be reported by Dependency Scanning analyzers as part of the Dependency Scanning report.

Once the sources are made available to the Rails backend, their URLs can be stored in a new packagesources table.

so that they can be reused across projects dependencies.

```
sql
CREATE TABLE packagesources AS (
  id integer NOT NULL,
  url varchar NOT NULL
);
```

A new packagesourceid column is added to track the source from which a dependency has been fetched.

This column might be NULL because there are cases where the analyzer cannot report the package source.

For instance, the analyzer might parse a lock file that doesn't provide this information.

```
sql
ALTER TABLE projectdependencies ADD COLUMN packagesourceid;
```

Similar to packages and package versions, package sources are upserted before upserting project dependencies for a completed pipeline.

Before upserting project dependencies when a CI pipeline finishes, the backend fetches the current package source IDs from the projectdependencies table and compares them to the source IDs of the corresponding dependencies it has in memory (and

about to be upserted).

It then creates vulnerability findings for each project dependency where the source is about to change,  
and eventually proceeds with the upsert.

### Reporting a package checksum mismatch

Users want to be warned when a project dependency matches a public package,  
but the checksum of the dependency doesn't match any distribution of that public package.

When upserting project dependencies after a pipeline completes,  
the backend collects the dependencies added to the project,  
and uses the package metadata DB to collect metadata on the public package versions matching these dependencies.

It then iterate the dependencies, and creates a vulnerability finding when the checksum of the dependency  
doesn't match any checksum of the corresponding package version.

New vulnerability findings created by the backend are false-positives in some cases:

- The dependency is a private package whose name and version match a public package.
- The dependency uses a package distribution or an alternative checksum that isn't tracked in the package metadata DB.

If the package source is being tracked in the projectdependencies table,  
then this can be leveraged to exclude private packages when comparing the checksums.

### Package metadata

Package metadata usually include the name of the package, a description of its purpose, an author list, a dependency list, etc.

Package metadata can be included in the installable package itself or kept in a separate file.  
Also, they might be repeated by the registry hosting the package, and be fetched using the registry API.

Package registries might provide:

- version metadata specific to a package version
- package metadata that applies to all versions

For instance, the [rubygems.org](https://rubygems.org) API provides both  
gem methods and  
gem version methods.

In most cases version metadata are immutable whereas package metadata are not.  
For instance, the description, authors, and license of a package might change over time,  
but these metadata won't change for a specific package version.

### Tracking Service

We introduce a new service that tracks public packages, looks for specific package metadata, and posts notifications when these metadata change.

Depending on the package registry, the tracking service will either poll the registry API or listen to a feed of events.

In a simple design, the service could track relevant metadata fields in a relational DB.

1. When a package is seen for the first time, the fields being tracked are store in a DB table.

1. When new metadata is available for the same package, these metadata are compared to what's in the DB.

1. If the new value doesn't match the old one, a notification is posted and the DB record is upserted.

If we decide to track a new field, then two metadata updates are required to detect a change of that field, even if the package is already tracked in the DB.

To remove that limitation, the service can store raw metadata coming from package registries in a document database (Object Storage).

When we decide to track a new field, we can process the existing metadata stored in the doc DB to fill up the new DB column corresponding to that field.

The service might use two DB tables to track changes in package metadata and version metadata, respectively.

These tables would be similar to the packages and packageversions tables of the GitLab backend. However, the backend only tracks packages and versions used as project dependencies, and it only stores the information it needs to search for specific dependencies.

The GitLab backend listens to the notifications posted by the tracking service and reacts by acting on the projects that depend on the package that's updated.

For instance, it might react by creating vulnerability findings in all projects using the updated package.

The tracking service also exposes API endpoints to fetch the fields it extracts from package metadata, acting as a proxy for the package registries it tracks.

### License Scanning outside of a pipeline

Users want to track licenses used by their project dependencies without running a CI pipeline. They are aware that running License Scanning outside of a pipeline has limitations, like not being able to track licenses used by private packages.

To perform License Scanning outside of a pipeline, the backend extracts package types, names, and versions from the project dependencies and uses the package Tracking Service API to collect the corresponding licenses. It then upserts licenses into softwarelicenses, and inserts records into projectdependencylicenses to reflect the licenses that have been found.

Doing License Scanning outside of a pipeline might be inaccurate if the project dependency is a private package

whose type, name, and version match a public package tracked by the Tracking Service. If the package source of project dependencies are tracked in the relational DB, then the backend can leverage this information to exclude private packages from the scan.

The backend can leverage the relational DB to list project dependencies if the branch being scanned is tracked by the projectdependencies table. If this is not the case, then it uses the Dependency Scanning reports or any other document that lists the project dependencies for the commit being scanned.

### Notifications about package updates

Users want to be notified about important events relative to packages their projects depend on. In particular, they want to be notified about these events:

- A new major version has been published.
- A new minor version of the major version being used has been published.
- A new patch version of the minor version being used has been published.
- The major version being used has reached end of life.
- The software license has changed.
- The package maintainers have changed.
- The maintainers of the upstream project have changed.

When one of these events occurs, the Tracking Service notifies the GitLab backend, and the backend reacts by listing all the projects using the updated package, and creating user notifications for these.

Depending on the event, the backend might exclude projects that are not affected by the package update.

For instance, it won't create notifications for a new minor version 1.2.3 if a project uses version 2.0.0 of the package.

Since creating notifications for all projects depending on an updated package is a long process, this task might be split up in multiple asynchronous jobs. For instance, there could be one job per namespace and/or one job per project.

The exact notification messages and how they are delivered to users are to be defined.

### Warnings about yanked packages

Users want to be warned when a package their projects depend on has been yanked from the package registry.

This warning shows up as a new vulnerability finding listed in the security dashboard.

When a package is yanked from a package registry, the Tracking Service notifies the GitLab backend, and the backend reacts by creating vulnerability findings in all projects that depend on the same package type and name.

Similarly, when it's notified about a yanked package version, the backend creates vulnerability findings in projects that depend on the exact same package

type, name, and version.

Since creating vulnerability findings for all projects depending on a yanked package is a long process, this task might be split up in multiple asynchronous jobs. For instance, there could be one job per namespace and/or one job per project.

The vulnerability finding created by the backend might be a false-positive when the package or version yanked from the public registry matches a private package the project depends on. However, the private package might be fetched from a registry that mirrors the public registry, so the finding might still be relevant.

### Package score

Some services provide metrics on public packages, like a score that measures the quality and health of a package. Like the package registries, they provide package metadata that can be tracked using the Tracking Service.

GitLab could either rely on existing package scoring services or create its own scoring service; in both cases the Tracking Service notifies the backend about a score change in the exact same way.

### Warnings about score going down

Users want to be notified when the score of a package they use as a project dependency goes down.

When the package score goes down (like from B to C), the Tracking Service notifies the GitLab backend, and the backend reacts by listing all the projects using the updated package, and creating user notifications for these. This is similar to sending notifications about a metadata that has changed on the package registry.

### Security advisories

A security advisory describes a vulnerability that applies to specific versions of a package. Unlike ordinary package metadata, security advisories associated to a given package version do change:

- A new security advisory can affect a package version at any time.
- An existing advisory might be updated. For instance, its CVSS score might change.
- An existing advisory might be removed because it no longer affects the version.

Security advisories can be provided by The GitLab Advisory Database, the vulnerability database maintained by GitLab, or by other vulnerability databases, like ruby-advisory-db.

## Tracking vulnerability databases

In order to support multiple vulnerability databases, we introduce a new service that tracks these databases and notifies listeners about two kinds of events:

- A security advisory has been added to a vulnerability database.
- An important field of an existing advisory has changed.

The tracking service also acts as a proxy for the vulnerability databases: it exposes an API endpoint that returns the security advisories for any package identified by a type, a name, and a version.

If GitLab only uses its own vulnerability database, then there are two possible architectures:

- The tracking service is built on top of the gemnasium-db git repo, and it uses web hooks to react in real-time when YAML files are added or modified.
- gemnasium-db is replaced by a service that hosts the security advisories. It behaves like the tracking service and notifies listeners about new advisories and advisory updates.

## Dependency Scanning outside of a pipeline

Users want to scan their project dependencies without running a CI pipeline. They are aware of limitations, like not being able to scan projects that have no lock files.

When running outside of a pipeline, Dependency Scanning relies on reports that list project dependencies.

These reports could either be the existing Dependency Scanning reports or some kind of SBoM artifacts.

In the future, the Rails backend could also process the lock files or dependency exports directly, removing the need for CI jobs that list project dependencies.

To perform Dependency Scanning outside of a pipeline, the backend proceeds in 4 steps:

1. extract the package types, names, and versions from the project dependencies being scanned
1. connect to the advisory database API or tracking service API to collect the advisories that match the package types and names
1. evaluates the affected version range of each advisory; dependencies that use a version that's not in range are not affected
1. create vulnerability findings for each affected dependency

When creating vulnerability findings, the backend combines:

- information specific to the affected dependency, like the package version and the file where the dependency is declared
- generic information coming from the security advisory, like a description of the security flaw, the CVE id, or the CVSS vector

In a mixed environment where Dependency Scanning can be performed by both the backend and scanning jobs,  
the backend must set the vulnerability finding fingerprint,  
so that it matches the fingerprint of the same vulnerability reported by a scanning job.  
If the fingerprints don't match, then the vulnerability detected by the backend  
will then be marked as no longer detected after a pipeline with Dependency Scanning jobs  
finishes.

Performing Dependency Scanning outside of a pipeline might lead to false-positives if the  
project dependency is a private package  
whose type and name match a public package for which there's a security advisory.  
However, this private package might be fetched from a registry that mirrors the public  
registry, so the vulnerability finding might still be relevant.

When running on the CI, Dependency Scanning can benefit from post-analyzers  
that cross check Dependency Scanning reports with other reports in order to flag false-  
positives.  
If that's the case, then Dependency Scanning won't benefit from these post-analyzers when  
running outside of a pipeline,  
and it might report false-positives.

The backend can leverage the relational DB to list project dependencies  
if the branch being scanned is tracked by the projectdependencies table.  
If this is not the case, then it uses the Dependency Scanning reports or any other document  
that lists the project dependencies for the commit being scanned.

When running outside of a pipeline, Dependency Scanning should support all the filters it  
supports when running on the CI.  
In particular, it should be capable of excluding development dependencies from the scan when  
this is supported by the Dependency Scanning CI job.  
Today scan filters are configured using CI variables, and these are not directly available to  
the Rails backend,  
so we might move scan filter settings to project settings.

### Warnings about new security advisories

Users want to be warned about new security advisories affecting their project dependencies  
without running a new CI pipeline.  
They receive a notification, and a new vulnerability finding appears in their security  
dashboard.  
They are aware that this finding might not be as accurate as the ones reported by a Dependency  
Scanning CI job,  
and that this might be a false-positive.

When a new security advisory is added to a vulnerability database, the GitLab backend is  
notified and it responds in 4 steps:

1. It lists all the projects using the affected package as a dependency, based on its package  
type and name.
1. It evaluates the range of the affected versions, and filters out the projects using a

package version that's not affected.

1. It creates vulnerability findings in all remaining projects.
1. Finally, it creates user notifications for these new vulnerabilities.

Since creating vulnerability findings and user notifications for all affected projects is a long process,

this task might be split up in multiple asynchronous jobs.

For instance, there would be one job per project that depends on the affected package.

The job would follow steps:

1. evaluate the affected version range, and exit if the project doesn't depend on any affected version
1. create vulnerability findings for all project dependencies that match the affected version range
1. create notifications for all users who have notifications enabled for that project

The backend uses the relational DB to efficiently list all the project dependencies that match the newly affected package.

### Notifications about updated security advisories

Users want to be notified when a security advisory affecting their project dependencies changes.

In particular, they want to know when the CVSS score changes or when a new fix is available.

When a new security advisory is updated, the GitLab backend is notified about that change, and it lists all the projects with a primary vulnerability identifier that matches the ID of the security advisory.

It then creates user notifications about this change in all matching projects.

It is assumed that the primary identifier of the Dependency Scanning vulnerability finding is the ID of the security advisory, like the CVE ID.

This identifier is tracked in the existing `primaryidentifierid` column of the `vulnerabilityoccurrences` table.

To find projects affected by the updated advisory, the backend performs a SQL query that joins `projects`, `vulnerabilityoccurrences` and `vulnerabilityidentifiers`.

The two last tables are joined on `vulnerabilityoccurrences.primaryidentifierid`.

If the advisory has a CVE ID, then query is filtered on identifiers `WHERE`

`vulnerabilityidentifiers.externaltype` is `cve`

and where `vulnerabilityidentifiers.externaltype` equals the CVE id.

The query that lists matching projects uses the existing index on `vulnerabilityoccurrences.primaryidentifier`.

To make it efficient, a composite index on `vulnerabilityidentifiers.externaltype` and `vulnerabilityidentifiers.externalname` is also needed.

### Software of Bill of Materials

A Software Bill of Materials (SBOM) is a document that lists components in a piece of software. The components are identified using URLs, sets of coordinates, or some other kind of IDs. Components might have additional metadata like a list of authors, URLs, licenses, security advisories, etc.

The document usually describes both the software for which the SBOM was created and the tool used to create the SBOM.

In this document we focus on application-level packages detected by Dependency Scanning and License Scanning

even though SBOMs can also list system-level packages detected by Container Scanning or components tracked manually and not detected by scanners.

There are many different SBOM formats but this document focuses on CycloneDX because it is comprehensive, extensible, and well supported by open source libraries and automation tools.

## CycloneDX

CycloneDX can be used for an inventory of components used as project dependencies.

CycloneDX components can be identified using SWID, CPE, or Package URL (PURL).

Like the package slug of the gemnasium-db vulnerability database,

the PURL combines the package type, the full package name, and the package version.

PURL also supports non-default package registries or repositories.

Components can be linked to describe a dependency graph.

Dependencies can either be transient (dependencies of components) or direct (dependencies of the software itself).

Dependencies have no metadata and it's not possible to distinguish development dependencies from production ones.

However, dependency metadata could be supported by extending the CycloneDX format.

Components can have multiple licenses.

A license can be described using a name, a URL, a SPDX identifier, and by quoting the license itself.

SPDX expressions are also supported.

Components can have external URLs,

and each URL can provide multiple checksums or "hashes".

This can be used to store signatures of package distributions.

SBOM metadata can describe the software for which the inventory was created.

It doesn't seem possible to describe a composite software made of multiple subprojects, like a Gradle multi-project build.

However, there are at least two ways the parent project and its subproject can be described using CycloneDX:

- by describing the subprojects as "components" of the parent project
- by having multiple CycloneDX documents

CycloneDX doesn't track the dependency files being used to do an inventory.

As a result, the SBoM can't be used to redirect users to the file that introduces a component listed in the inventory.

However, there are ways we can circumvent this limitation:

- Use global metadata properties to track the dependency file (and optional lock file) where the dependencies were detected. It's not possible to track multiple files of the same type though.
- Use component properties to track the dependency file (and optional lock file) where each dependency was detected. This introduces redundancies but is more flexible.
- Create a CycloneDX extension to accurately describe the dependency files, and how they relate to the components.

The vulnerability extension

can be used to embed vulnerabilities in the SBoM and link them to the components.

A vulnerability can have:

- a CVE identifier
- a description
- a source name (like NVD) and URL
- CVSS vectors, base score, etc.
- CWE identifiers
- advisory URLs
- remediations

A vulnerability only has one identifier, and it looks like this has to be a CVE ID. To be checked.

Also, it doesn't seem possible to provide a short description for the vulnerability.

## Internal SBoM

The Internal SBoM (ISBoM) is a document or a set of documents that describe all the dependencies of a project commit.

It must contain all the data needed to generate SBoM exports, render the Dependency List, compare dependencies, and perform new dependency scans on old commits.

The primary goal of the ISBoM is to reduce maintenance cost by introducing a file format that is both comprehensive and extensible;

this way we don't have to maintain adapters for a variety of formats and format versions.

Newer versions of the Rails backend should remain backwards compatible with older versions of the ISBoM,

making it possible to export a new SBoM for an old git commit without running a new CI pipeline.

To achieve that goal, and considering that the SBoM represents a lot of information, the ISBoM is stored as a file (or files) in Object Storage, and not in the relational DB.

Also, the file paths or URLs should be predictable so that the ISBoM can be retrieved without storing a significant amount of information in the relational DB.

To that effect, the ISBoM could either be:

- a single SBoM document that can track multiple dependency files and describe a multi-project repository

- a collection of SBoM documents combined in an archive

The secondary goals are:

- Fast parsing: Project dependencies are efficiently extracted from the ISBoM, with or without graph information. It should be possible to extract project dependencies corresponding to a set of dependency files without parsing the entire ISBoM.
- Efficient Storage: The ISBoM is compressed when it becomes unlikely that this is going to be accessed.

Since it's a long lasting document, the ISBoM should only contain immutable data. In particular, it might contain license information but it should not contain vulnerabilities detected in project dependencies; over time vulnerabilities can change and new vulnerabilities can be added, resulting in a vulnerability list that's inaccurate and/or incomplete.

The ISBoM is primarily used to create SBoM exports but it can also be used to support other features when project dependencies are not tracked in the relational DB:

- compare dependencies between the source branch and the target branch of an MR
- render the Dependency List
- show dependency paths or dependency graphs in the Dependency List
- find new vulnerabilities

For features that only need basic information on project dependencies, the ISBoM can be used as a fallback when project dependencies are not available in the relational DB for the commit being processed.

(It's assumed that it's faster to retrieve these information from the DB tables.)

In this case, the ISBoM can be used to fill the DB tables, to later use these tables as a cache.

If the scanning jobs don't provide the information needed to export the SBoM, then this information should be retrieved from external services and added to the ISBoM. For instance, if license information or the authors list is needed for the SBoM export, they should be fetched from the registry or repository hosting the package, either directly or via a Tracking Service.

This complicates the creation of the ISBoM but ensures that the ISBoM is complete and can be used directly.

SBoM as a License Scanning artifact

CycloneDX is a superset of the License Scanning (LS) report format:

- Combined with the package manager, the package name and version (LS) can be converted to a Package URL (CycloneDX).
- License IDs (LS) can be used as SPDX IDs (CycloneDX) if they match the SPDX License List.
- License can have a name and a URL in both formats.

Like CycloneDX, LS reports don't track the dependency files that introduce project dependencies

or the subprojects of a multi-project build/repository.

As a result, the existing License Scanning job can be changed to generate CycloneDX documents. Conversely, the Rails backend can be changed to use CycloneDX documents instead of the existing LS reports or to support both formats.

However, the fields the backend needs to implement License Scanning features are optional in CycloneDX

and the backend must be able to handle incomplete documents.

### SBoM as a Dependency Scanning artifact

CycloneDX is NOT a superset of the Dependency Scanning (DS) report format:

- It can't track the dependency file that introduces a dependency, either directly or indirectly.
- It can't describe a composite project, like multi-project Gradle build.
- Vulnerabilities can't have a short description (vulnerability name).
- Vulnerabilities can't have alternative identifiers, and it looks like only CVE IDs are supported.

However, it's possible to remove these limitations by extending CycloneDX.

(It might be necessary to generate multiple CycloneDX documents to accurately describe multi-project builds and repositories.)

As a result, the existing Dependency Scanning job can be changed to generate CycloneDX documents,

but these documents won't be as accurate as the original DS report until we write the CycloneDX extensions that fill the gap.

The Rails backend can be changed to use CycloneDX documents instead of the existing DS reports, but not before extending CycloneDX so that its SBoM accurately tracks dependency files.

We might change the backend to support vulnerabilities that don't have a short description and alternative identifiers; this seems affordable.

However, the backend heavily relies on the dependency file path when rendering the dependency list and when tracking vulnerabilities.

### SBoM exports

Users want to export the SBoM of their projects to the format they use, like CycloneDX.

They might need an SBoM that covers all the projects of a project group, a workspace, or an instance.

If it takes a significant amount of time to generate the SBoM, users need to be notified when it's ready.

Depending on project settings and group settings, SBoM exports are created:

- on user request, for any project commit
- whenever a project release is created
- periodically, for all relevant branches

Technically an SBoM export of a project commit could be created from:

- all the Dependency Scanning and License Scanning reports of the latest successful pipeline
- the Internal SBoM corresponding to the commit

Using the ISBoM as the input has multiple benefits:

- There's no need to query the CI database to list the CI artifacts.
- There's only one file to retrieve from Object Storage.
- The ISBoM format is more stable compared to the reports used to build it.
- The ISBoM already provides package metadata needed for the SBoM export.
- Overall this is faster, especially when processing a large number of projects.

The SBoM export may consist of multiple SBoM documents combined in an archive.

- The export might cover all the projects of a project group or a workspace, resulting in multiple SBoM documents.
- The GitLab project might be a multi-project repository, and depending on the SBoM format it might be necessary to create multiple SBoM documents to do a full inventory.

When the exports consists of multiple SBoM documents, it is available as a `tgz` archive or a `zip` archive, depending on the project settings or group settings.

Compressed archives are convenient to download and they also have a positive impact on storage.

For simplicity, on-demand exports could be limited to a single project to ensure that it only takes seconds to generate the export. If quick enough, this task could be handled synchronously, which simplifies the UI and removes the need for a dedicated Rails worker.

It's assumed that SBoM exports of a project group or workspace take too long to be performed synchronously on demand request, and that these tasks should be handled by dedicated Rails workers. Periodic SBoM exports are then performed by periodic Rails jobs.

SBoM exports created asynchronously are stored in Object Storage. Users might need an SBoM export longer after the corresponding code was published.

- If SBoM exports are created from CI artifacts, then they should have a specific retention policy, and expire later.
- If SBoM exports are created from long lasting ISBoMs, then they can be safely removed to save space.

SBoM as a release asset

One or multiple SBoM exports are linked to a project release when it's created, depending on the project settings.

The SBoM exports are stored on Object Storage and linked to the release using asset links. Ideally the feature supports three scenarios:

- The SBoM export is already available when creating the release.
- The SBoM export is in progress, and linked when available.

- There is no SBoM export. The export should be triggered and the SBoM should later be linked when available.

Explain dependencies using graphs

Users want to know why their project depends on a specific package even though it's not explicitly required;

they want to know the relationship between a transient dependency and dependencies introduced in the dependency files they maintain.

This relationship could be presented in different ways:

- list all the paths that connect the transient dependency to the introduced ones
- show the full dependency graph and highlight nodes and edges that connect the transient dependency to the introduced ones
- show a sub-graph that only includes the transient dependency and the introduced dependencies connected to it

A single project might have multiple dependency graphs, and these might be stored in multiple artifacts generated by one or multiple scanning jobs.

- Depending on the report format being used, multiple documents might be necessary to describe all the dependencies of a multi-project repository or build.

- When a repository uses multiple package managers, it might be processed by multiple scanning jobs, resulting in multiple CI artifacts.

That said, users only want to know how one specific transient dependency is connected to introduced dependencies,

and a single dependency graph is sufficient to answer that question;

the information can be retrieved from a single Dependency Scanning report or SBoM artifact.

The implementation is mostly the same independently of what is presented in the UI.

1. The frontend asks the sub-graph for a package name, package version, dependency file path, commit SHA, and project ID.

1. If the sub-graph is highlighted in the graph, then the frontend also requests the full graph for a file, commit, and project.

1. The backend fetches the CI artifact corresponding to the file, commit, and project.

1. The backend builds the full dependency graph and calculates the sub-graph.

1. The backend returns the sub-graph, a list of paths, or whatever the frontend needs to explain the transient dependency.

Frontend could use GraphQL to query the graph.

If users explore one specific transient dependency, it becomes likely that they explore other transient dependencies of the same file, commit, and project.

As an optimization, the backend should then cache the corresponding dependency graph, so that these steps are not repeated:

1. List the CI artifacts and find the one corresponding to a dependency file, commit, and project.

1. Parse the CI artifact being found and extract graph information.

Alternatively, dependency graph information could be fetched from the ISBoM.

If that's the case, then the ISBoM format should be optimized to efficiently retrieve the graph for a specific dependency file

without parsing and loading in memory the entire ISBoM.

Group vulnerabilities by introduced dependencies

Users want to know which dependencies introduced in the editable dependency files are responsible for the vulnerabilities being detected.

To address this, the existing dependency list should group vulnerabilities per introduced dependency.

An introduced dependency is responsible for a vulnerability if it's directly affected or it depends on a package that's affected (a transient dependency).

Transient dependencies might be shared by multiple introduced dependencies, so there's a N-N relationship between introduced dependencies and vulnerabilities:

- An introduced dependency might have multiple vulnerabilities.
- A vulnerability might be connected to multiple introduced dependencies.

Grouping vulnerabilities by introduced dependencies could be extended to all dependencies, that is any transient dependency that has dependencies.

For instance, if an introduced dependency A depends on B, and transient dependency B depends on C and D, then we could group:

- vulnerabilities related to A; these are vulnerabilities directly affecting A, B, C, or D
- vulnerabilities related to B; these are vulnerabilities directly affecting B, C, or D

The relationships between the affected dependencies and their ancestors are described by the dependency graph.

The dependency graph is not part of the Dependency Scanning report at the moment, but the existing report will be replaced, extended, or completed to provide the graph.

To efficiently fetch the introduced dependencies from the relational DB, we could add a new introduced boolean column to the projectdependencies table, and set when upserting project dependencies for the branch being tracked.

The backend fetches and processes the dependency graph to establish whether a dependency is introduced or transient.

The column is NULL if the graph isn't available. This might be because of limitations in the analyzer or in the dependency files they parse.

sql

```
ALTER TABLE projectdependencies ADD COLUMN introduced boolean;
```

To prepare a list of introduced dependencies along with all related vulnerabilities, the backend proceeds in 4 steps:

1. fetch introduced project dependencies from the relational DB, possibly using projectdependencies.introduced
1. load corresponding dependency graphs from Object Storage
1. fetch vulnerability findings for the same project branch
1. build a dependency map that connects the affected dependencies to the introduced dependencies
1. group the vulnerability findings per introduced using the dependency map

The dependency list could be limited to the dependencies introduced in one specific file, like api/Gemfile.

This would make the backend more efficient:

- It fetches the project dependencies matching that file when querying the relational DB.
- It fetches all vulnerability findings for the project branch, but it doesn't keep in memory the findings where the location doesn't match the file.
- It only loads and processes the dependency graph corresponding to the file.

Technically, api/Gemfile could be resolved to api/Gemfile.lock if this is the only file we track in the projectdependencyfiles table.

The dependency map used to resolve affected transient dependency to introduced dependencies could be cached for performance.

The scope of the cache would be the project commit and the set of dependency files for which the map was generated.

---

## SECTION: engineering/development/sec/secure/tech-docs/feedback.md

## Overview

Once a Finding is reported for a project, users can interact with it in multiple ways. One of them is called Feedback and allows the user to:

- dismiss the finding
- create an issue from the finding
- create an MR from the finding

These features are described in details in our user documentation.

It has been called Feedback as the initial intent was to gather feedback from users about reported Findings and possibly leverage that to increase the signal to noise ratio.

## Challenges

One challenge with the Feedback is the reliability of its tracking.

Indeed, when a Finding is dismissed, we want to make sure it will stay dismissed in any subsequent scan that would report it.

Another one is that the Feedback feature is available on all views, for all branches.

This means, it must work with the Vulnerability Findings (that are stored in the database) and the Report Findings (that are NOT stored in the database).

## Implementation

NB: there is currently a proposal to replace that implementation.

The Vulnerability::Feedback AR model relies on a projectfingerprint attribute to match it with a Finding. This approach allows to:

1. have a stable identifier between scans to keep matching the feedback with the finding over time and over different branches
1. match feedback from any source (Database or JSON artifact) as this doesn't rely on DB only feature like foreign key.

Though, due to how we generate this fingerprint, 1 is not as stable as expected and is also not generated in a consistent way between the different analyzers.

This projectfingerprint is stored in the database for Vulnerability Finding records and dynamically generated when parsing Report Findings from JSON artifacts (for MR widget and Pipeline view).

The Feedback records are scoped at the project level as they are meant to be shared between all branches, hence the name "project" fingerprint.

Only one Feedback of a given type (dismissal, issue or mergerequest) is allowed for a given projectfingerprint and report type.

Even if that fingerprinting data originates from the Scanner output, it is not given in that final form in the JSON artifact.

How the projectfingerprint is generated

```
mermaid
flowchart LR
    subgraph "Scanner"
        A(foo)
    end
    subgraph "Analyzer"
        B(cve)
    end
    subgraph "Parser"
        C(comparekey)
    end
    subgraph "Finding"
        D(projectfingerprint)
    end
    Scanner-->Analyzer
    Analyzer-->Parser
    Parser-->Finding
```

For each Finding, the Analyzer creates a cve property from proprietary data given by the Scanner and that may vary from one Scanner to another.  
So unlike its name, this property doesn't contain a CVE value anymore.  
This legacy property was initially used for comparison and deduping of findings in the early stage of the security features, which was only based on CVE number at that time.  
Then the property has been overloaded in an attempt to provide a value that will achieve both uniqueness and stability over time, and in a backward compatible way.  
The cve property is part of the Secure Report Format.

When parsing the report in the rails application, this cve property is unmarshalled into a comparekey attribute of the Finding object.

This comparekey is then hashed into a SHA1, which is assigned to the projectfingerprint attribute.

If the Finding is stored in the database, the projectfingerprint is saved there too in the corresponding column.

How the projectfingerprint is used

When retrieving Findings, we can fetch associated Feedback by looking for the corresponding projectfingerprint, whatever the data source the Findings are coming from.

```
mermaid
graph LR
  subgraph "Database"
    A["(Vulnerability Finding)"]
    B["(Feedback)"]
  end
  subgraph "JSON Artifact"
    C["Report Finding"]
  end
  subgraph "Logic"
    A --> M1
    B --> M1
    M1["Match projectfingerprint"]
    M2["Match projectfingerprint"]
    B --> M2
    C --> M2
  end
  subgraph "Views"
    M1 --> X["Dashboards"]
    M2 --> Y["MR & Pipeline view"]
  end
```

---

## SECTION: engineering/development/sec/secure/vulnerability-research/\_index.md

<p align="center">  
  
</p>

## Vision

Facilitate the Application Security Testing stage achieving lovable maturity and supporting the best possible security assessment at the earliest possible moment.

Customer outcomes we are driving for GitLab

Vulnerability Research sits at the crossroads between the Application Security Testing stage itself, and customers of the stage. We provide enhancements to our products or processes to ensure our products and services are more effective for GitLab's customers.

We strive to enhance our customer experience with regards to providing correct and accurate results from our services.

## Vulnerability Research Team

{{< team-by-departments departments="Secure:Vulnerability Research BE Team" >}}

## Priorities

Vulnerability Research (VR) is a research & development team. VR contributes to GitLab's business by working with product groups to improve GitLab's security products. Advanced security is a key driver of GitLab Ultimate revenue.

Our priorities are:

1. Analyze and improve the quality of the results of GitLab's existing security products, including:

- Assessing current quality against industry-standard or custom benchmarks.
- Identifying the types of work required to improve quality.
- Creating new detection rules or curating existing detection rules to improve detection efficacy.

1. Conduct research experiments that are designed to advance the maturity of GitLab security product categories, including:

- Short-term bets, aligned and coordinated with product groups.
- Long-term bets, aligned and coordinated with Application Security Testing stage leadership.

1. Maintain and evolve the GitLab Advisory Database, which is used in GitLab Dependency Scanning and Container Scanning.

1. Carry out CVE Numbering Authority (CNA) duties. GitLab is a CNA.

## Common Links

- Slack channel: [gast-vulnerability-research](#)
- GitLab group: [gitlab-org/secure/vulnerability-research](#)

## Mission

Our mission is to advance GitLab's security offerings toward their long-term vision and to elevate GitLab's prominence in the security community. We do this by performing security research, working to improve the efficacy of GitLab's security capabilities, developing proofs of concept, publishing papers, speaking at conferences, and broadly sharing our security expertise and practical experience.

## How we work

### Ideate

- Type of improvement: DevSecOps adoption, Revenue, Security Quality/Efficacy, Cost, Efficiency
- Product orientation: how the product can best benefit from this idea?
- Evaluation criteria: how will we measure progress and success?
- Reviewers: Application Security Testing Product Managers and Engineering Managers
- Publication potential: can we present, blog about, or patent this work?
- Tracking: We use the group::vulnerability research label on [gitlab-org/gitlab](#) issues as our SSOT

These types of initiative are tracked using the `vulnerability research::initiative` labels, according to the nature of the initiative:

- Product related initiatives are tracked with `vulnerability research::initiative::product`
- Independent research related initiatives are tracked with `vulnerability research::initiative::independent`

### Prioritize

- Success rate: Balance short-term and long-term bets
- Timing: POC projects should always be aligned with short- or long-term product vision (as appropriate), should be developed in consultation with the product team that will eventually inherit it, and an anticipated timeline for that expected handover should be jointly understood.
- Publication: Potential for using as a source for something public

### Prototype

- Timebox: Short-term bets should be mature enough within three months to decide whether to continue them. The time frame for long-term bets is within six months.
- Customers & prospects: Identify those outside our team to help validate the prototypes (including coordinating with PM and UXR)

### Build and Ship

The vulnerability researcher's top priority is, as needed, to support the engineering team until the feature becomes available in GitLab. A secondary focus is to publish things publicly

(blogs, talks, etc.).

## Metrics

Metrics we are working on:

- Reported rule false positive rate - in flight
- Mean time to publish advisory database updates - in discussion
- Mean time to respond to publish new CVEs - in discussion

## Rule Updates

The group focuses on improving the precision and recall of our rulesets; the former by reducing false positives and the latter by reducing false negatives.

To report an inaccuracy (false positive) or a missed result (false negative), open a new issue using the following templates:

- Report a False positive
- Report a False negative

Work related to this effort is tracked in this Issue Board. Issues are labeled using the following:

- False positives should have the labels `AST::Ruleset::FP` and `type::maintenance`
- False negatives should have the labels `AST::Ruleset::FN` and `type::feature`
- Both issue types should also set the category label, for example `Category:SAST`.

We prioritize rule updates and additions based on product management direction, which includes taking into account maximum customer value such as these factors:

- Severity of the rule, because:
  - Customer workflows are more likely to be disrupted by high-severity rules.
  - False-negative results are more concerning if they fail to report a higher-severity vulnerability.
- Customer escalations about the rule or rules due to inaccurate or missing findings
- Benchmarking results and platform-wide data analyses with a focus on false positive rates

We prioritise our work by applying priority labels ranging from 1 to 4, with the former being the highest priority and the latter the least.

## Workflow label usage

The team uses these workflow labels for issues:

- Status Open - The issue is not vetted nor prioritized. Vulnerability Research (VR) with collaboration with PM as needed will prioritize then add the label `workflow::ready for development`
- `workflow::refinement` - The issue needs further input from the team in order to be ready for development.

- workflow::ready for development - The issue is ready for a researcher to start work.
- workflow::in dev - A researcher is working on it and has assigned the issue to themselves and added this label.
- workflow::blocked - Work cannot continue on this due to an outside dependency. This can be set by anyone collaborating on the issue, generally the assignee. When using this comment as to why the issue is blocked or link the blocking issue.
- workflow::in review - The MRs associated with the issue are in review by the picked and assigned reviewer(s). This label is applied by assignees to the issue once the development is complete.
- workflow::complete - The MRs associated with the issue are all merged. The issue assignee sets this label.
- Status: Closed - The MRs associated with the issue is incorporated into a release that is available to customers. The issue assignee sets this state.

---

## SECTION: engineering/development/sec/secure/vulnerability-research/cna.md

This handbook page is intended to document CNA processes that the Vulnerability Research team uses in contributing to GitLab's role as a CNA.

## CVE Requests

### Events we operate upon

The team will operate upon a CVE Publish Request issue being marked as "ready for Immediate Publishing".

### Workflow

1. Triage: Within 2 days of receiving the request, we review the issue for completeness and accuracy.

2. De